Parallel Finite Difference Method for the Poisson Problem (Preview: Only Pages 1 and 2)

H. Noah Belzer

December 6, 2024

Abstract

This report explores the numerical solution of the Poisson problem on a unit square domain using the Finite Difference Method (FDM) and parallel computing. By implementing the solution in C, both serial and parallel computations can be run to approximate the answer and measure performance against varying data sizes. The implementation uses Message-Passing Interface (MPI) to distribute the computation, specifically through blocking and non-blocking communication, which leads to improved efficiency. Performance comparisons reveal significant speedups for the parallel C code. The non-blocking case and blocking case gave very similar results, however, the non-blocking implementation showed improvement, particularly at larger grid sizes, making it the most scalable and efficient approach.

Table of Contents

1 Introduction **2**

2 Methods and Implementation **3**
- 2.1 Serial C: Code Improvement . . . 3
- 2.2 Parallel C: Code Improvement . . . 6
- 2.3 Parallel C: Using Non-Blocking MPI . . . 7

3 Results **7**
- 3.1 Convergence of the Finite Difference Method . . . 8
- 3.2 Parallel Performance Study with Blocking MPI_Send/MPI_Recv . . . 9
- 3.3 Parallel Performance Study with Non-Blocking MPI_Isend/MPI_Irecv . . . 10

4 Conclusions **17**

Acknowledgments **18**

References **18**

Appendix **19**

1 Introduction

Problem Statement

The primary goal of this report is to approximate the elliptic test problem of the Poisson equation on the unit square domain $\Omega = (0,1) \times (0,1) \subset \mathbb{R}^2$ and perform simulations for moderately fine mesh resolutions, such that they are large enough to fully test numerical convergence.

Poisson Equation and the Finite Difference Method

The Poisson Equation is given by

$$\begin{aligned} -\Delta u &= f(x,y) \text{ for all } (x,y) \in \Omega = (0,1) \times (0,1) \subset \mathbb{R}^2 \\ u &= 0 \text{ for all } (x,y) \in \partial\Omega \end{aligned} \tag{1}$$

The Laplace operator is defined as

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \tag{2}$$

We will check $x_0 = y_0 = 0$ and $x_{N+1} = y_{N+1} = 1$ on the boundary.

Additionally, the function f is given by

$$f(x,y) = -2\pi^2 \cos(2\pi x)\sin^2(\pi y) - 2\pi^2 \sin^2(\pi x)\cos(2\pi y) \tag{3}$$

A closed-form solution as the true solution takes the form

$$u(x,y) = \sin^2(\pi x)\sin^2(\pi y) \tag{4}$$

We are aware that it is important to compute the finite difference error to confirm the convergence of the Finite Difference Method. The finite difference error is defined as the difference between the true solution and the numerical solution that is defined on the mesh points. We know that for sufficiently small h, we can expect that the ratio of errors on consecutively refined meshes will behave like

$$\text{Ratio} = \frac{||u - u_{2h}||}{||u - u_h||} \approx \frac{C(2h)^2}{Ch^2} = 4 \tag{5}$$

Our code implements a matrix-vector product:

$$\vec{v}_{ij} = -\vec{u}_{ij-1} - \vec{u}_{i-1j} + 4\vec{u}_{ij} - \vec{u}_{i+1j} - \vec{u}_{ij+1} \tag{6}$$

Furthermore, in Tables 3.1.1 and 3.1.2 in the Results section, we will provide a column for the computed ratio values. Lastly, the correct norm for the theory of finite differences is the $L^\infty\Omega$ function norm, which is defined by

$$||u - u_h||_{L^\infty(\Omega)} = sup_{(x,y)\in\Omega}|u(x,y) - u_h(x,y)| \tag{7}$$

We compiled the results with blocking and non-blocking MPI to determine that the non-blocking MPI cases are faster at higher nodes, although the overall runtimes were similar.